

3-Day Workshop Plan — Team Task Tracker (React + NestJS + PostgreSQL)

Objective:

Build a full-stack web application using React, NestJS, and PostgreSQL that supports authentication, task management, and file upload with inline PDF/image viewing. Participants can use AI tools (ChatGPT, GitHub Copilot, etc.) for assistance.

Tech Stack

- **Frontend:** React + TypeScript + Tailwind CSS (or MUI)
 - **Backend:** NestJS + TypeORM + PostgreSQL
 - **Testing:** Unit tests on backend APIs
 - **CI/CD:** Not included
-

Day 1 – Setup & Core Backend

Goal: Set up environment, configure PostgreSQL, and implement user authentication.

Topics & Activities:

- Overview of architecture (React ↔ NestJS ↔ PostgreSQL)
- Environment setup: `.env`, DB connection, Postman setup
- Create NestJS project and connect with PostgreSQL via TypeORM
- Define **User** entity: `id`, `name`, `email`, `password_hash`, `role`
- Implement **Register** and **Login** APIs using JWT
- Add validation, error handling, and password hashing
- Initialize React project with Vite + TypeScript

Deliverables:

- Backend Auth module completed (Register/Login)
 - React app initialized and running
 - PostgreSQL schema created and verified
-

Day 2 – Task Module, File Upload & Frontend Integration

Goal: Build the Task management module, enable file uploads, and connect frontend.

Topics & Activities:

- Backend:

- Create **Task** entity: `id`, `title`, `description`, `status`, `due_date`, `assigned_to`, `file_url`
- Implement CRUD endpoints for tasks
- Add **file upload API** using Multer (store images/PDFs)
- Serve static files from `/uploads`
- Add unit tests for `TaskService` and `AuthService`
- Frontend:
 - Implement login/register UI and store JWT
 - Create Task list and add/edit task forms
 - Integrate file upload field in Task form
 - Add inline viewer for uploaded files:
 - Images: `<img>` preview
 - PDFs: `<iframe>` or viewer component

Deliverables:

- Authenticated users can perform CRUD on tasks
- File upload feature working with inline preview
- At least one backend unit test running successfully

Day 3 – Review, Enhancement & Demo

Goal: Finalize, review, and present the application.

Activities:

- Code cleanup and documentation (README)
- Run backend unit tests and verify API responses
- Optional enhancements using AI:
 - Task search/filter
 - Status badges or visual indicators
 - Improved form validation and UX
- Instructor-led review:
 - Code structure, modularity, and API design
 - TypeScript usage and code readability
- Demos and feedback session

Deliverables:

- Fully functional app with file upload and viewer
- Reviewed and documented codebase
- Feedback collected from participants

Summary:

Area	Deliverable
Backend	Auth & Task APIs with file upload (NestJS + PostgreSQL)
Frontend	React UI for login, task CRUD, and file viewer (image/PDF)
Testing	Backend unit tests
Docs	README with setup instructions
Review	Demo and feedback session on Day 3